

1. Prosjektbeskrivelse

Formålet er å utvikle en enkel, men skalerbar **multi-tenant bookingportal** basert på samme teknologistack som kandidat 1:

- Hver kunde (tenant) registrerer seg, velger virksomhetstype (f.eks. hytteutleie, frisør) og får sin egen underside (slug).
- Kunden kan definere sine **booking-objekter** (hytter, frisørstoler, behandlere, rom osv.) med tilhørende tilgjengelighet i kalender.
- Systemet har **rollebasert tilgang**:
 - Kunde (tenant-admin / brukere hos kunden) med eget dashboard.
 - System-admin med oversikten over alle kunder, abonnement og innstillinger.
- **Abonnementssystem** uten betaling i første fase:
 - Bruker velger abonnement.
 - Middleware sjekker om brukeren har aktivt abonnement.
- Integrasjon mot **Teletopia SMS**:
 - Admin kan lagre API-nøkkel for SMS i databasen.
 - Mulighet for å sende testsms fra admin for å verifisere integrasjon.
- Landingsside som viser alle registrerte kunder, med lenke til hver kundes egne bookingside via slug.

Kandidaten er juniorutvikler under Jobbloop med faglig veiledning, og skal aktivt bruke AI-verktøy i arbeidet (planlegging, kodeforslag, dokumentasjon og feilsøking), men alltid forstå, kvalitetssikre og teste det som leveres.

2. Tech Stack

Prosjektet skal bygges på:

- **Laravel 12**
 - Auth, routing, Eloquent, migrasjoner, middleware, policies, osv.
 - Multi-tenant-støtte via egne tabeller og tenant_id-felt (ikke avansert SaaS-rammeverk i denne fasen).

- **Laravel Breeze**

- Grunnleggende pålogging/registrering.
- Kan utvides med roller og abonnement.

- **Alpine.js**

- Lettvekts interaktivitet: dynamiske skjemaer, modaler, kalender-interaksjon på klienten.

- **Tailwind CSS**

- Styling av landingsside, dashboards, booking-views og admin-grensesnitt.

All konfigurasjon og alle innstillinger som brukerne kan påvirke (Teletopia-API-nøkkel, abonnementer, virksomhetstype, bookingoppsett etc.) lagres i databasen. Det skal ikke hardkodes nøkler eller kundespesifikke data i .env (unntak: systemets egne interne nøkler hvis nødvendig, men ikke per-tenant).

3. Hovedleveranser

3.1. Multi-tenant struktur og abonnement

- **Modellering av tenants/kunder:**

- Tabell for tenants/customers med felter som navn, virksomhetstype, slug, aktiv-status.
- Relasjon mellom users og tenants (f.eks. tenant_id på bruker eller pivot-tabell hvis flere tilknytninger).

- **Abonnement:**

- Tabell for plans (standard abonnementstyper).
- Tabell for subscriptions som knytter tenant til plan, inkl. aktiv-flag/datoer.

- **Middleware:**

- Middleware som sjekker om innlogget bruker tilhører en tenant med aktivt abonnement før tilgang til booking/dashboard.

- **Registreringsflyt:**

- Utvidet registreringsprosess:

- Opprette bruker.
- Opprette tenant.
- Velge plan.
- Aktivere tjenesten (f.eks. sett `subscription.active = true`).

3.2. Booking-objekter og kalender

- **Booking-objekter** (ressurser):
 - Tabell for resources (f.eks. hytte, stol, rom) med `tenant_id`.
 - Felter som navn, beskrivelse, kapasitet, type.
- **Tilgjengelighet / kalender:**
 - Enkel modell for tilgjengelighet, f.eks.:
 - `resource_availabilities` med dato, fra-til, eventuelt mønster (daglig/ukentlig).
 - Alternativt enkel struktur: tilgjengelig alle dager 08–16 i første versjon.
- **Bookings:**
 - Tabell for bookings knyttet til resource, dato/tid og kundeinformasjon (sluttbruker som booker).
- **Kunde-underside (slug):**
 - Offentlig rute for `/ {slug}` som viser:
 - Oversikt over ressursene til den aktuelle tenant.
 - Booking-skjema der eksterne kunder kan velge dato/tid innenfor tilgjengelighet og registrere en booking.

3.3. Teletopia SMS-integrasjon

- **Innstillinger i database:**
 - Tabell for `tenant_settings` eller egen `sms_settings`-tabell med Teletopia API-nøkkel per tenant.
- **Admin-funksjon:**
 - Admin for tenant må kunne:

- Lagre/endre Teletopia API-nøkkel.
- Sende test-SMS fra admin-grensesnittet:
 - Input: telefonnummer.
 - Output: tilbakemelding om at SMS er sendt/feilet.
- **SMS-tjeneste:**
 - Egen service-klasse (f.eks. TeletopiaSmsService) som håndterer kall mot API.
 - Klar til å brukes i framtidige funksjoner (f.eks. booking-bekreftelser).

3.4. Rollebasert tilgang og dashboards

- **Roller:**
 - Minst to roller:
 - admin (systemadministrator).
 - customer / tenant_admin (kundeadmin hos hver tenant).
 - Evt. underroller for ansatte hos kundene på sikt.
- **Dashboards:**
 - **Kunde-dashboard:**
 - Oversikt over abonnement.
 - Status på konto.
 - Antall registrerte booking-objekter.
 - Snarveier til å administrere ressurser, tilgjengelighet og SMS-innstillinger.
 - **System-admin-dashboard:**
 - Liste over alle tenants.
 - Oversikt over abonnementer.
 - Enkle innstillinger pr tenant (aktiv/deaktiv, etc.).

3.5. Landingsside med kundeliste og slug-lenker

- Landingsside (/) som:

- Presenterer systemet (kort tekst).
 - Viser oversikt over alle registrerte og aktive kunder (tenants).
 - Viser lenker til hver kundes underside via slug, f.eks. /salong-rosa der man kan booke direkte.
 - Evt. enkel filtrering/søk på landingssiden (valgfritt hvis tid).
-

4. 2-ukers fremdriftsplan (10 arbeidsdager)

Uke 1 – Struktur, registrering, abonnement og grunnleggende multi-tenancy

Dag 1 – Oppstart, miljø og domeneforståelse

- Oppgaver:
 - Introduksjon til prosjekt, multi-tenant-konsept og ønsket funksjonalitet.
 - Sette opp Laravel 12 + Breeze + Tailwind + Alpine.
 - Opprette Git-branch for prosjektet.
- AI-bruk:
 - Generere sjekkliste for oppsett.
 - Få forslag til mappestruktur for multi-tenant-løsning (uten å introdusere unødvendig kompleksitet).

Dag 2 – Modellering av tenants, brukere og planer

- Oppgaver:
 - Designe tabeller og relasjoner for:
 - tenants (kunder).
 - plans.
 - subscriptions.
 - Lage migrasjoner, modeller og grunnleggende Eloquent-relasjoner.
- AI-bruk:
 - Generere forslag til migrasjonsfiler og modeller.

- Få hjelp til å formulere fornuftige felter (f.eks. slug, active_from, active_to, status).

Dag 3 – Registrering og opprettelse av tenant + abonnement

- Oppgaver:
 - Utvide Breeze-registreringen:
 - Når en ny bruker registrerer seg, opprettes samtidig en tenant med valgt virksomhetstype og slug.
 - Opprette første subscription (uten betaling) og markere som aktiv.
 - Enkelt skjema i registreringsflyt for å velge virksomhetstype (hytteutleie, frisør, etc.) og plan.
- AI-bruk:
 - Hjelp til å utvide Breeze sine kontrollere og views.
 - Forslag til validering og slug-generering (Str::slug etc.).

Dag 4 – Middleware for aktivt abonnement + grunnleggende rolle-oppsett

- Oppgaver:
 - Implementere middleware:
 - Sjekker om innlogget bruker har tenant med aktiv subscription.
 - Redirecte/vis beskjed hvis ikke aktiv.
 - Innføre roller:
 - Evt. role-felt på users eller egen roles-tabell (enkel løsning i første omgang).
- AI-bruk:
 - Generere eksempel-middleware.
 - Forslag til enkel RBAC-struktur innenfor tidsrammen.

Dag 5 – Dashboards (kunde og admin) – første versjon

- Oppgaver:
 - Lage ruter og views for:

- **Kunde-dashboard:**
 - Vise informasjon om tenant, abonnement, virksomhetstype.
 - **Admin-dashboard:**
 - Liste over alle tenants og deres abonnement.
 - Enkel, oversiktlig layout med Tailwind.
 - AI-bruk:
 - Få forslag til Tailwind-layout for dashboards.
 - Generere Blade-partials for å gjenbruke komponenter (kortkomponenter, tabeller).
-

Uke 2 – Booking-objekter, kalender, SMS og landingsside

Dag 6 – Modellering av booking-objekter og tilgjengelighet

- Oppgaver:
 - Definere tabeller:
 - resources (booking-objekter, knyttet til tenant_id).
 - resource_availabilities eller tilsvarende for tilgjengelighet.
 - Lage migrasjoner, modeller og relasjoner.
- AI-bruk:
 - Hjelp til å velge enkel, men utvidbar modell for tilgjengelighet.
 - Generere forslag til relasjoner og Eloquent-scopes.

Dag 7 – CRUD for booking-objekter og enkel kalenderlogikk

- Oppgaver:
 - Admin-grensesnitt for tenants:
 - Opprett/rediger/slett booking-objekter.
 - Enkel kalender-/tilgjengelighetslogikk:
 - F.eks. definere åpningstider pr. dag, eller minimum: definere hvilke dager ressursen er tilgjengelig.

- AI-bruk:
 - Generere eksempel på et ressurs-admin-UI i Blade + Tailwind.
 - Hjelp til å implementere enkel kalender-/datalogikk i kontrollere eller services.

Dag 8 – Kunde-underside (slug) og booking-skjema

- Oppgaver:
 - Offentlig rute: /{slug}:
 - Slår opp tenant via slug.
 - Viser liste over tilknyttede booking-objekter.
 - Implementere et booking-skjema som:
 - Lar sluttbruker velge ressurs og dato (eventuelt tid).
 - Lagrer booking i bookings-tabellen.
 - Viser en enkel bekreftelse til sluttbruker.
- AI-bruk:
 - Generere routing- og controller-logikk for slug-baserte sider.
 - Forslag til validering og enkel konflikt-sjekk (ikke dobbeltbook samme ressurs/tidspunkt).

Dag 9 – Teletopia SMS-integrasjon og innstillinger i database

- Oppgaver:
 - Lage tabell for sms_settings eller tilsvarende som inneholder:
 - tenant_id.
 - API-nøkkel og eventuelle ekstra parametre.
 - Admin for tenant:
 - Skjema for å lagre/oppdagere Teletopia-API-nøkkel.
 - Funksjon for å sende test-SMS til valgt nummer.
 - Implementere TeletopiaSmsService som:
 - Leser API-nøkkelen fra databasen.

- Utfører API-kall mot Teletopia (testmetode).
- AI-bruk:
 - Få forslag til service-klasse og enkel error-håndtering.
 - Hjelp til å skrive pseudo-kall mot Teletopia basert på dokumentasjon (kandidaten legger inn faktiske endpoints).

Dag 10 – Landingsside, opprydding, tester og presentasjon

- Oppgaver:
 - Implementere landingsside (/) som:
 - Forklarer tjenesten kort.
 - Viser liste over aktive tenants (kunder) med lenke til /{slug}.
 - Opprydding:
 - Rydde i routes, kontrollere og views.
 - Kommentere og kortdokumentere hoveddeler av koden.
 - Enkle tester:
 - Manuelle tester av:
 - Registrering og tilknytning til tenant.
 - Abonnement-middleware.
 - Opprettelse av booking-objekter.
 - Booking via slug-side.
 - Test-SMS med Teletopia.
 - Forberedelse av kort demo/presentasjon:
 - Flyt: fra landingsside → registrering → dashboard → booking-setup → slug-side → booking → test-SMS.
 - AI-bruk:
 - Få forslag til struktur i README (oppsett, funksjonsoversikt).
 - Generere skisse til demo-manus, punktvis.
-

5. Bruk av AI – forventninger og retningslinjer

Kandidaten skal:

- Bruke AI til å:
 - Bryte ned oppgaver i mindre steg.
 - Få kodeforslag til migrasjoner, modeller, kontrollere, Blade-views og services.
 - Få forklaringer på konsepter: multi-tenancy, middleware, role-based access, API-integrasjon.
 - Hjelp til å skrive korte beskrivelser og dokumentasjon.
- Alltid:
 - Lese gjennom og forstå AI-generert kode.
 - Tilpasse kode til prosjektets konvensjoner og krav.
 - Teste funksjonalitet før commit.

Veileder skal:

- Kvalitetssikre kritiske deler (autentisering, autorisasjon, multi-tenant-isolasjon).
 - Gi tilbakemelding på arkitekturvalg og bruk av AI (balanse mellom læring og effektivitet).
-

6. Suksesskriterier

Overordnet

- Kandidaten leverer en fungerende prototype på en multi-tenant bookingportal, hvor:
 - Bruker kan registrere seg, opprette tenant og aktivere abonnement.
 - Middleware håndhever at kun aktive abonnement har tilgang til bookingfunksjon.
 - Tenants kan opprette booking-objekter med tilgjengelighet.
 - Sluttbrukere kan gå til kundens slug-side og gjøre booking.
 - Teletopia-integrasjon er implementert med test-SMS-funksjon.
 - Alle relevante innstillinger (inkl. Teletopia API-nøkkel) ligger i databasen.

Spesifikke suksesskriterier

1. Multi-tenancy og abonnement

- tenants, plans, subscriptions er korrekt modellert og fungerer i praksis.
- Middleware avviser/redirecter brukere uten aktivt abonnement.

2. Booking og kalender

- Ressurser (booking-objekter) kan opprettes pr tenant.
- Enkle regler for tilgjengelighet fungerer.
- Bookings lagres og vises i et oversiktlig grensesnitt.

3. Teletopia SMS

- API-nøkkel lagres i database, ikke i .env.
- Test-SMS kan sendes fra admin-UI og gir tydelig tilbakemelding.

4. Rollebasert tilgang og dashboards

- Minst to roller med forskjellig tilgangsnivå.
- Kunde-dashboard og admin-dashboard er funksjonelle og informative.

5. Landingsside og slug-baserte undersider

- Landingssiden viser liste over tenants med lenker til deres slug.
- Sluttbruker kan gå til /{slug}, se ressurser og gjøre booking.

6. Læring og AI-bruk

- Kandidaten viser forståelse for multi-tenant-prinsipper, rollebasert tilgang og enkel SaaS-arkitektur.
- Kan peke på konkrete situasjoner der AI hjalp, og situasjoner der AI-forslag måtte korrigeres.